ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL

REDES NEURONALES III

Optimización, regularización y evaluación de modelos de Deep Learning

Informe técnico del taller · Semanas 5, 6 y 7

Tabla 1. Datos de la actividad

Campo	Información
Actividad	Redes Neuronales III (herramienta de recolección: Documento)
Estudiante(s)	Juan Nicolás Romero Estupiñan
Programa	Especialización en Inteligencia Artificial
Asignatura	Deep Learning - Conceptos
Docente	Nidia Stella García Roa
Fecha	octubre de 2026
Repositorio GitHub	https://github.com/NicolasRomero22/Redes-Neuronales-III

Contenido

1. Introducción
2. Marco conceptual
3. Datos y preprocesamiento
4. Modelo baseline
5. Optimización e hiperparámetros
6. Regularización y overfitting
7. Evaluación del modelo
8. Diagnóstico de bias y variance
9. Conclusiones
10. Repositorio en GitHub
Referencias

1. Introducción

El presente informe documenta el desarrollo de la actividad **Redes Neuronales III**, cuyo propósito es avanzar desde una red que aprende ajustando pesos hacia una red cuyo entrenamiento pueda ser analizado, mejorado y evaluado con evidencia. Para ello se aplican técnicas de **optimización**, **regularización** y **evaluación** sobre un mismo problema de clasificación multiclase —el reconocimiento de dígitos manuscritos del conjunto MNIST—, de modo que las diferencias observadas entre experimentos puedan atribuirse al cambio de optimizador, hiperparámetro o técnica de regularización y no al uso de datos distintos.

El trabajo se desarrolló en Google Colab con TensorFlow + Keras, scikit-learn para la partición de datos y el cálculo de métricas, pandas para la organización de resultados y matplotlib para las gráficas. La progresión es intencional: primero se organizan los datos (train/dev/test), luego se construye una línea base, se comparan optimizadores y learning rates, se controla el overfitting con Dropout, L2 y Early Stopping, y finalmente se evalúa el modelo con métricas que van más allá del accuracy.

Objetivos específicos

- Particionar MNIST en Train, Dev y Test de forma estratificada, normalizar las entradas y verificar la distribución de clases.
- Construir un modelo baseline y analizar sus curvas de loss y accuracy.
- Comparar los optimizadores SGD con momentum, RMSprop y Adam, y explorar el efecto del learning rate.
- Aplicar Dropout, regularización L2 y Early Stopping, y medir su efecto sobre la brecha Train–Dev.
- Evaluar el modelo final con Accuracy, Precision, Recall, F1 (promedio macro) y matriz de confusión, analizando errores concretos.
- Diagnosticar situaciones de bias alto, variance alta o equilibrio a partir de Train y Dev.

Organización del notebook

Todas las prácticas se desarrollaron en un único notebook de Google Colab, **TallerRedesNeuronalesIII.ipynb**, organizado por laboratorios en el orden de la guía. La práctica opcional en PyTorch no se desarrolló; se cubre con TensorFlow + Keras, que la actividad admite.

Tabla 2. Laboratorios del notebook y evidencia principal

Laboratorio	Contenido	Evidencia principal
A. Datos y baseline	Train/dev/test, normalización, distribución y baseline.	Shapes, distribución, summary, curvas y accuracy.
B. Optimización	SGD, RMSprop, Adam y learning rate.	Tabla comparativa y análisis.
C. Regularización	Sin regularización vs. Dropout + L2 + Early Stopping.	Curvas train/dev y época de parada.
D. Métricas	Accuracy, Precision, Recall, F1 y matriz de confusión.	Tabla de métricas y errores.

Laboratorio	Contenido	Evidencia principal
E. Bias y variance	Modelo pequeño, sin regularización y regularizado.	Tabla de diagnóstico Train–Dev.

Cada sección de este informe indica el laboratorio y el notebook de origen de sus capturas y de su código fuente.

2. Marco conceptual

2.1 Parámetros e hiperparámetros

Los **parámetros** son los valores que la red aprende durante el entrenamiento (pesos y sesgos) y que el optimizador modifica a partir de los gradientes. Los **hiperparámetros** son decisiones de configuración que definen cómo se realiza el aprendizaje y que fija quien diseña el experimento: learning rate, batch size, número de épocas, número de neuronas, tasa de dropout y fuerza de la penalización L2. El learning rate no es un peso: es el tamaño del paso con el que el optimizador actualiza los pesos.

Tabla 3. Parámetros frente a hiperparámetros

Tipo	Ejemplos	Quién los modifica
Parámetros	Pesos y bias	El optimizador, a partir de los gradientes.
Hiperparámetros	Learning rate, batch size, épocas, neuronas, dropout, fuerza L2	La persona que diseña y prueba el experimento.

2.2 Conjuntos Train, Dev y Test

Train actualiza los pesos; **Dev** (validación) permite comparar configuraciones y monitorear la generalización sin actualizar pesos; **Test** se reserva para la evaluación final y no debe utilizarse para escoger optimizador, learning rate ni arquitectura. Usar Test repetidamente para elegir configuraciones constituye **data leakage**: equivale a estudiar con las preguntas del examen final y produce estimaciones de desempeño optimistas.

2.3 Optimización

Tabla 4. Optimizadores comparados

Optimizador	Idea	Qué observar
SGD + Momentum	Sigue el gradiente y conserva parte de la dirección anterior.	Puede requerir más ajuste del learning rate.
RMSprop	Adapta el paso según el historial reciente de gradientes cuadrados.	Puede comportarse bien cuando cambian las escalas.
Adam	Combina momentum y adaptación del paso (primer y segundo momento).	Buen punto de partida, no un ganador universal.

2.4 Regularización

Tabla 5. Técnicas de regularización

Técnica	Qué hace	Implementación en el taller
Dropout	Desactiva aleatoriamente una proporción de unidades durante el entrenamiento.	layers.Dropout(0.30)
L2	Penaliza pesos excesivamente grandes (suma de cuadrados).	regularizers.l2(1e-4)
L1	Penaliza el valor absoluto de los pesos y los favorece cercanos a cero (extensión conceptual).	regularizers.l1(·) — no usada en los experimentos
Early Stopping	Detiene el entrenamiento cuando val_loss deja de mejorar.	patience=2, restore_best_weights=True

2.5 Bias, variance y tradeoff

Tabla 6. Lectura de la brecha Train–Dev

Situación	Patrón típico	Interpretación
Bias alto / underfitting	Train bajo y Dev también bajo.	El modelo no captura suficiente patrón.
Variance alta / overfitting	Train alto y Dev claramente inferior.	El modelo se adapta demasiado a Train.
Mejor equilibrio	Train y Dev altos con brecha moderada.	El modelo parece generalizar mejor.
Tradeoff	Reducir bias puede aumentar variance y viceversa.	Se busca equilibrio, no una cifra aislada.

2.6 Métricas de clasificación

Tabla 7. Métricas de evaluación

Métrica	Pregunta que responde	Definición
Accuracy	¿Qué proporción total fue correcta?	Aciertos / total
Precision	Cuando el modelo predice una clase, ¿con qué frecuencia acierta?	TP / (TP + FP)
Recall	De los ejemplos reales de una clase, ¿cuántos encontró?	TP / (TP + FN)
F1 Score	¿Cómo equilibrar Precision y Recall?	2·P·R / (P + R) (media armónica)
Matriz de confusión	¿Qué clases se confunden entre sí?	Filas: clase real; columnas: clase predicha

En MNIST se emplea el **promedio macro**: la métrica se calcula por cada dígito y luego se promedia otorgando el mismo peso a cada clase, lo que evita que las clases más frecuentes dominen el resultado.

2.7 TensorFlow + Keras y PyTorch

Keras ofrece una API de alto nivel en la que `model.fit()` encapsula el ciclo de entrenamiento (incluida la retropropagación) y los callbacks, como EarlyStopping, permiten intervenir en él. PyTorch expone un ciclo de entrenamiento explícito: tras calcular la pérdida se invoca `loss.backward()` y los pesos se actualizan con `optimizer.step()`. Los fundamentos son los mismos; cambia el nivel de control sobre el ciclo.

Tabla 8. Equivalencias entre frameworks

TensorFlow/Keras	PyTorch	Idea común
layers.Dropout(0.30)	nn.Dropout(0.30)	Desactivar unidades durante Train.
regularizers.l2(1e-4)	weight_decay=1e-4	Regularización L2.
optimizer=Adam(...)	optim.Adam(...)	Actualizar parámetros.
model.fit(...)	Ciclo for explícito	Ciclo de entrenamiento.
Callback EarlyStopping	Control manual o librería auxiliar	Detener según validación.

3. Datos y preprocesamiento

Notebook de origen: TallerRedesNeuronalesIII.ipynb. Se utiliza el dataset MNIST, que contiene imágenes en escala de grises de 28 × 28 píxeles de dígitos manuscritos (clases 0 a 9). Es un problema ya conocido de la actividad anterior, de modo que el reto se centra en mejorar y evaluar el entrenamiento y no en comprender un dataset nuevo.

3.1 Importación de herramientas y semillas

Se importan las librerías y se fija la semilla (SEED = 42) en NumPy y TensorFlow para favorecer la reproducibilidad. Pueden persistir pequeñas diferencias entre entornos o hardware.

Código 1. Importaciones y semillas

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers, callbacks

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
```

3.2 Carga de MNIST

Código 2. Carga del dataset

```
(x_train_full, y_train_full), (x_test, y_test) = keras.datasets.mnist.load_data()

print("Train original:", x_train_full.shape, y_train_full.shape)
print("Test:", x_test.shape, y_test.shape)
```

Se esperan 60.000 imágenes de entrenamiento originales y 10.000 de test, todas de 28 × 28 píxeles.

3.3 Partición estratificada Train/Dev/Test

Del conjunto de entrenamiento original se separa un 20 % para Dev con `stratify=y_train_full`, que conserva proporciones de clase semejantes entre Train y Dev. El conjunto de Test original de 10.000 imágenes permanece intacto y reservado para la evaluación final. La partición resultante es de aproximadamente 48.000 imágenes para Train, 12.000 para Dev y 10.000 para Test.

Código 3. División Train/Dev

```
x_train, x_dev, y_train, y_dev = train_test_split(
    x_train_full, y_train_full,
    test_size=0.20,
    random_state=SEED,
    stratify=y_train_full
)

print("Train:", x_train.shape, y_train.shape)
print("Dev:", x_dev.shape, y_dev.shape)
print("Test:", x_test.shape, y_test.shape)
```

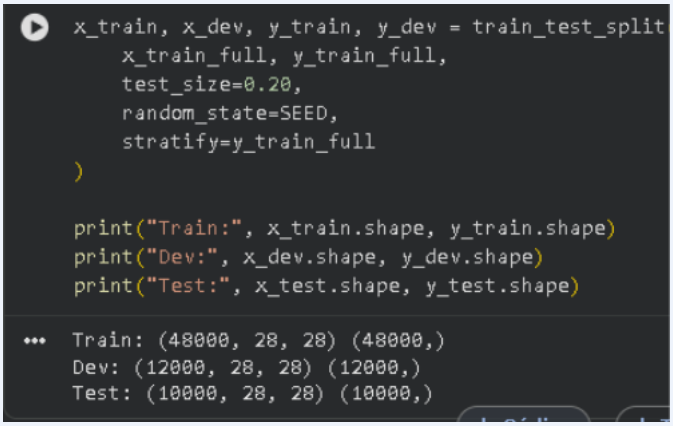


Figura 1. Formas (shapes) de los conjuntos originales, Train, Dev y Test

Qué muestra: Las dimensiones de cada conjunto tras la carga y la partición estratificada: Train 48.000, Dev 12.000 y Test 10.000 imágenes de 28 × 28 (a partir de 60.000 de entrenamiento originales y 10.000 de test).

Qué significa: Los tres conjuntos son disjuntos y de tamaños coherentes con la proporción 80/20; Test no participó en la partición.

Decisión que permite: Confirmar que se puede entrenar con Train, comparar configuraciones con Dev y reservar Test para el final.

3.4 Normalización

Código 4. Normalización de píxeles a [0, 1]

```
x_train = x_train.astype("float32") / 255.0
x_dev   = x_dev.astype("float32") / 255.0
x_test  = x_test.astype("float32") / 255.0

print("Rango train:", x_train.min(), x_train.max())
```



Figura 2. Rango de valores de los píxeles tras la normalización

Qué muestra: Los valores mínimo y máximo del conjunto Train tras dividir entre 255.

Qué significa: Las entradas pasan de la escala 0–255 a 0–1; la etiqueta del dígito no cambia, solo la escala de entrada, lo que facilita un entrenamiento más estable.

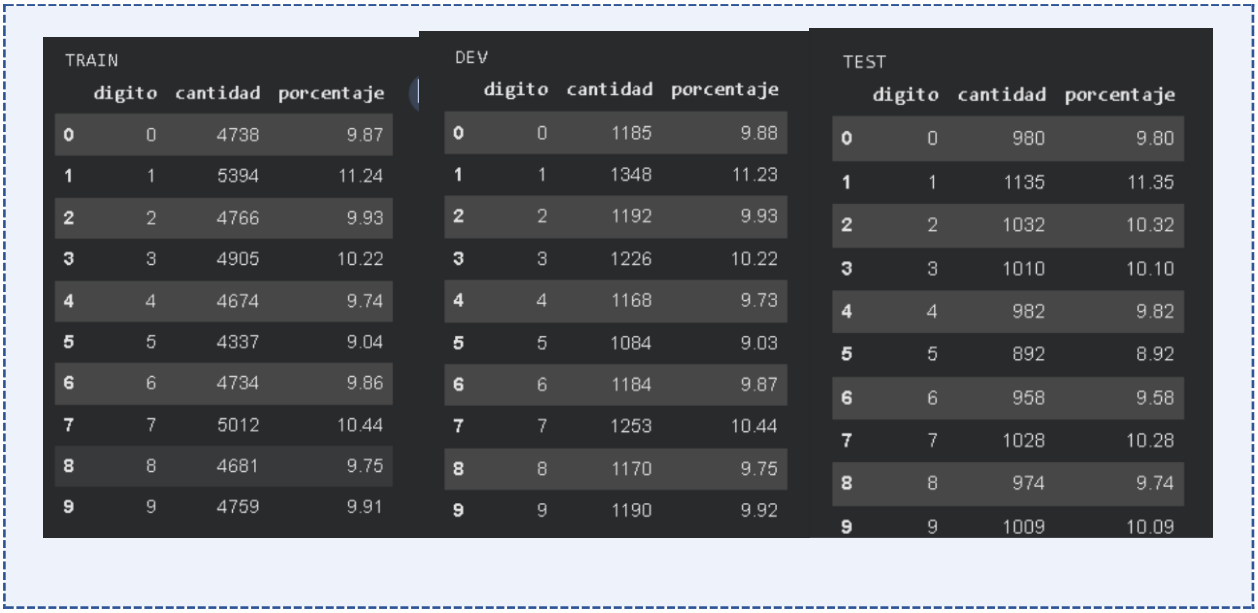
Decisión que permite: Aplicar la misma transformación a Dev y Test para que los tres conjuntos compartan escala.

3.5 Distribución de clases

Código 5. Verificación de la distribución de clases

```
def distribucion_clases(y, nombre):
    conteos = np.bincount(y, minlength=10)
    porcentajes = conteos / len(y) * 100
    tabla = pd.DataFrame({
        "digito": np.arange(10),
        "cantidad": conteos,
        "porcentaje": np.round(porcentajes, 2)
    })
    print("\n", nombre)
    display(tabla)
    return tabla

dist_train = distribucion_clases(y_train, "TRAIN")
dist_dev   = distribucion_clases(y_dev, "DEV")
dist_test  = distribucion_clases(y_test, "TEST")
```



TRAIN

	digito	cantidad	porcentaje
0	0	4738	9.87
1	1	5394	11.24
2	2	4766	9.93
3	3	4905	10.22
4	4	4674	9.74
5	5	4337	9.04
6	6	4734	9.86
7	7	5012	10.44
8	8	4681	9.75
9	9	4759	9.91

DEV

	digito	cantidad	porcentaje
0	0	1185	9.88
1	1	1348	11.23
2	2	1192	9.93
3	3	1226	10.22
4	4	1168	9.73
5	5	1084	9.03
6	6	1184	9.87
7	7	1253	10.44
8	8	1170	9.75
9	9	1190	9.92

TEST

	digito	cantidad	porcentaje
0	0	980	9.80
1	1	1135	11.35
2	2	1032	10.32
3	3	1010	10.10
4	4	982	9.82
5	5	892	8.92
6	6	958	9.58
7	7	1028	10.28
8	8	974	9.74
9	9	1009	10.09

Figura 3. Tablas de distribución de clases en Train, Dev y Test

Qué muestra: La cantidad y el porcentaje de ejemplos de cada dígito en los tres conjuntos.

Qué significa: Están presentes los diez dígitos en los tres conjuntos. Las proporciones de Train y Dev son prácticamente idénticas (por ejemplo, el dígito 1 representa 11,24 % en Train y 11,23 % en Dev, y el dígito 5, 9,04 % y 9,03 %), lo que confirma que la estratificación funcionó. Test difiere levemente (dígito 1: 11,35 %; dígito 5: 8,92 %), pero ningún conjunto presenta una distribución extraña: todas las clases oscilan entre ≈8,9 % y ≈11,4 %, sin desbalance severo.

Decisión que permite: Verificar que no existe un desbalance severo que obligue a ponderar clases y que la estratificación cumplió su función.

4. Modelo baseline

Notebook de origen: TallerRedesNeuronalesIII.ipynb. El baseline es una red densa pequeña que sirve como punto de referencia para todos los experimentos posteriores.

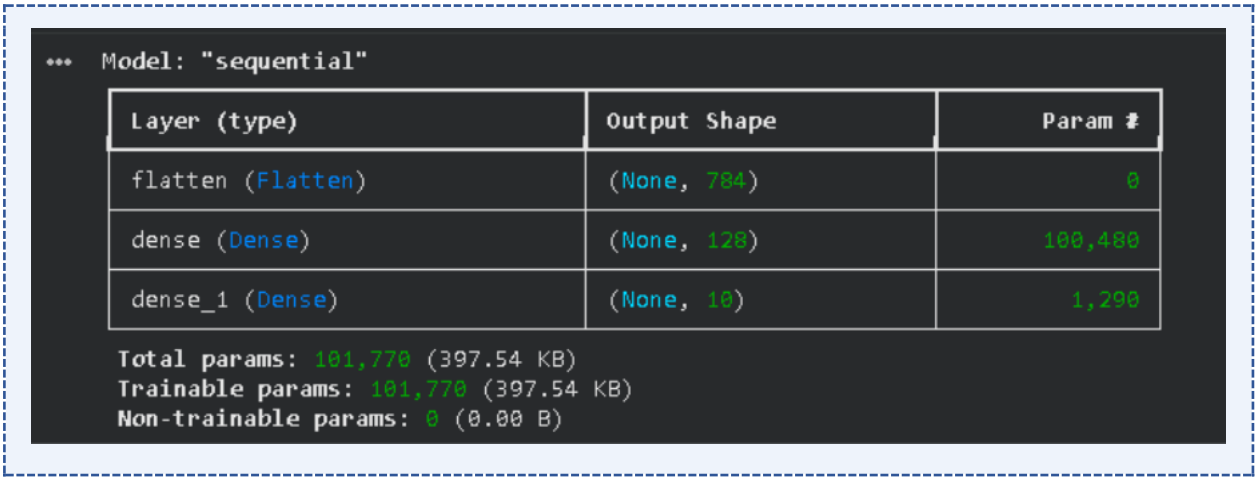
Tabla 9. Arquitectura y configuración del baseline

Elemento	Valor
Entrada	Imagen 28 × 28
Capas	Flatten (784) → Dense(128, ReLU) → Dense(10, Softmax)
Parámetros entrenables	101.770 (100.480 en la capa oculta y 1.290 en la salida)
Optimizador	Adam, learning rate = 0,001
Función de pérdida	sparse_categorical_crossentropy
Épocas / batch size	5 / 64
Datos de validación	Dev explícito: validation_data=(x_dev, y_dev)

Código 6. Definición del modelo baseline

```
def crear_modelo_baseline(units=128, optimizer=None):
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(units, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    if optimizer is None:
        optimizer = keras.optimizers.Adam(learning_rate=0.001)
    modelo.compile(
        optimizer=optimizer,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return modelo

modelo_base = crear_modelo_baseline()
modelo_base.summary()
```



Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

Figura 4. Resumen de la arquitectura del baseline (model.summary())

Qué muestra: Las capas del modelo, la forma de salida de cada una y el número de parámetros.

Qué significa: Casi todos los parámetros se concentran en la capa oculta (784 × 128 pesos más 128 sesgos).

Decisión que permite: Definir la capacidad de referencia frente a la cual se comparan los modelos más grandes (sección 6) y el modelo pequeño (sección 8).

4.1 Entrenamiento con Dev explícito

Train actualiza los pesos; Dev solo permite observar el desempeño en datos no utilizados para actualizar parámetros.

Código 7. Entrenamiento del baseline

```
hist_base = modelo_base.fit(
    x_train, y_train,
    validation_data=(x_dev, y_dev),
    epochs=5,
    batch_size=64,
    verbose=1
```

```
)
```

```
Epoch 1/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9068 - loss: 0.3359 - val_accuracy: 0.9446 - val_loss: 0.1937
Epoch 2/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9569 - loss: 0.1507 - val_accuracy: 0.9577 - val_loss: 0.1479
Epoch 3/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9696 - loss: 0.1056 - val_accuracy: 0.9618 - val_loss: 0.1296
Epoch 4/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9771 - loss: 0.0800 - val_accuracy: 0.9645 - val_loss: 0.1209
Epoch 5/5
750/750 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9818 - loss: 0.0629 - val_accuracy: 0.9657 - val_loss: 0.1165
```

Figura 5. Registro de entrenamiento del baseline (5 épocas)

4.2 Curvas de loss y accuracy

Código 8. Curvas del baseline

```
plt.figure(figsize=(7, 4))
plt.plot(hist_base.history["loss"], label="Train")
plt.plot(hist_base.history["val_loss"], label="Dev")
plt.xlabel("Época"); plt.ylabel("Loss")
plt.title("Baseline - Loss"); plt.legend(); plt.grid(True); plt.show()

plt.figure(figsize=(7, 4))
plt.plot(hist_base.history["accuracy"], label="Train")
plt.plot(hist_base.history["val_accuracy"], label="Dev")
plt.xlabel("Época"); plt.ylabel("Accuracy")
plt.title("Baseline - Accuracy"); plt.legend(); plt.grid(True); plt.show()
```

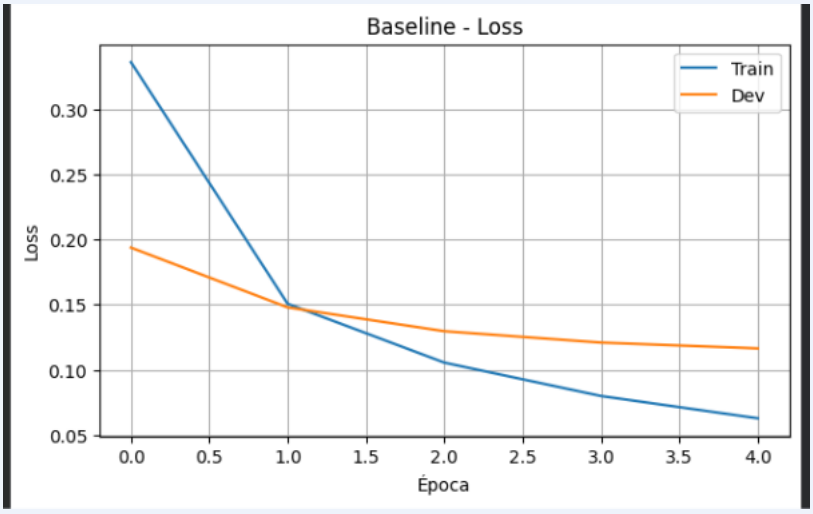


Figura 6. Curva de Loss del baseline: Train frente a Dev

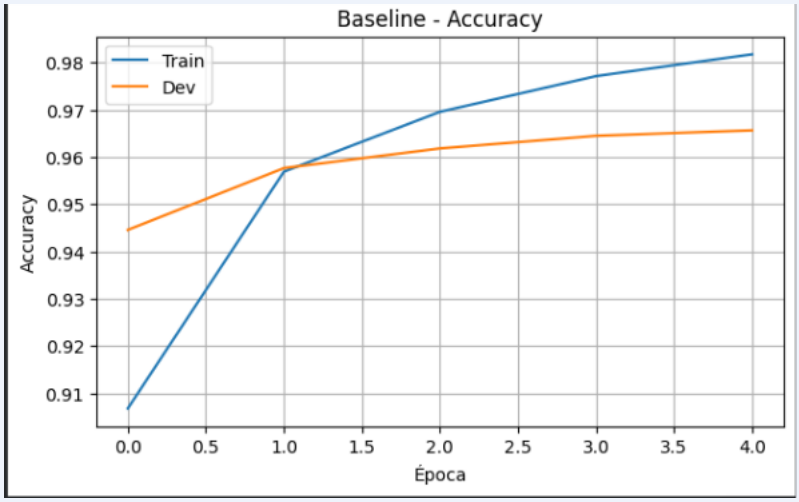


Figura 7. Curva de Accuracy del baseline: Train frente a Dev

Qué muestra: La evolución del loss y del accuracy en Train y Dev durante las cinco épocas. Resultado final: accuracy Train 0,9818 (loss 0,0629) y accuracy Dev 0,9657 (loss 0,1165).

Qué significa: El loss de Train y de Dev descienden en las cinco épocas, pero a ritmos distintos: el accuracy de Train sube de 0,9068 a 0,9818, mientras que el de Dev pasa de 0,9446 a 0,9657 y su mejora se vuelve cada vez menor (+0,0027 en la última época). La brecha de accuracy entre Train y Dev es de ≈1,6 puntos porcentuales y la de loss de ≈0,05. Dev no empeora, por lo que no hay overfitting marcado, pero la distancia creciente indica que el modelo empieza a ajustarse más a Train que a datos nuevos.

Decisión que permite: Establecer el valor de referencia del baseline y decidir si el siguiente paso es optimizar (sección 5) o regularizar (sección 6).

5. Optimización e hiperparámetros

Notebook de origen: TallerRedesNeuronalesIII.ipynb. Se comparan SGD con momentum, RMSprop y Adam manteniendo constantes el dataset, la arquitectura, el número de épocas (5) y el batch size (64), de modo que **solo cambia el método de optimización**. Antes de cada entrenamiento se limpia la sesión de Keras y se reinicia la semilla, y se crea una instancia nueva de optimizador por modelo para evitar errores por reutilización.

5.1 Comparación de optimizadores

Código 9. Función que crea el mismo modelo para cada experimento

```
def crear_modelo_experimento(optimizador, units=128):
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(units, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    modelo.compile(
        optimizer=optimizador,
```

```
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return modelo
```

Código 10. Comparación de SGD con momentum, RMSprop y Adam

```
fabricas_optimizadores = {
    "SGD_momentum": lambda: keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    "RMSprop":      lambda: keras.optimizers.RMSprop(learning_rate=0.001),
    "Adam":         lambda: keras.optimizers.Adam(learning_rate=0.001)
}

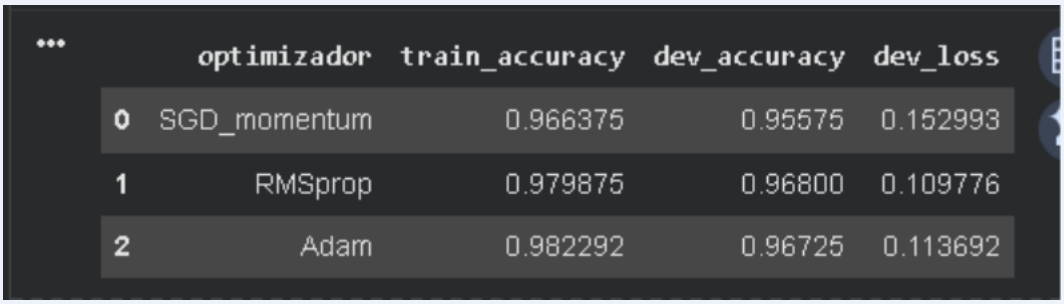
resultados_opt = []
historias_opt = {}

for nombre, fabrica in fabricas_optimizadores.items():
    keras.backend.clear_session()
    tf.random.set_seed(SEED)

    modelo = crear_modelo_experimento(fabrica())
    historia = modelo.fit(
        x_train, y_train,
        validation_data=(x_dev, y_dev),
        epochs=5, batch_size=64, verbose=0
    )

    historias_opt[nombre] = historia
    resultados_opt.append({
        "optimizador": nombre,
        "train_accuracy": historia.history["accuracy"][-1],
        "dev_accuracy": historia.history["val_accuracy"][-1],
        "dev_loss": historia.history["val_loss"][-1]
    })

tabla_opt = pd.DataFrame(resultados_opt)
display(tabla_opt)
```



	optimizador	train_accuracy	dev_accuracy	dev_loss
0	SGD_momentum	0.966375	0.95575	0.152993
1	RMSprop	0.979875	0.96800	0.109776
2	Adam	0.982292	0.96725	0.113692

Figura 8. Tabla comparativa de optimizadores (train_accuracy, dev_accuracy, dev_loss)

Tabla 10. Resultados de la comparación de optimizadores

Optimizador	Learning rate	Train accuracy	Dev accuracy	Dev loss
SGD con momentum (0,9)	0,01	0,9664	0,9558	0,1530

Optimizador	Learning rate	Train accuracy	Dev accuracy	Dev loss
RMSprop	0,001	0,9799	0,9680	0,1098
Adam	0,001	0,9823	0,9673	0,1137

Código 11. Validation loss por optimizador

```
plt.figure(figsize=(8, 5))
for nombre, historia in historias_opt.items():
    plt.plot(historia.history["val_loss"], label=nombre)
plt.xlabel("Época"); plt.ylabel("Validation loss")
plt.title("Comparación de optimizadores")
plt.legend(); plt.grid(True); plt.show()
```

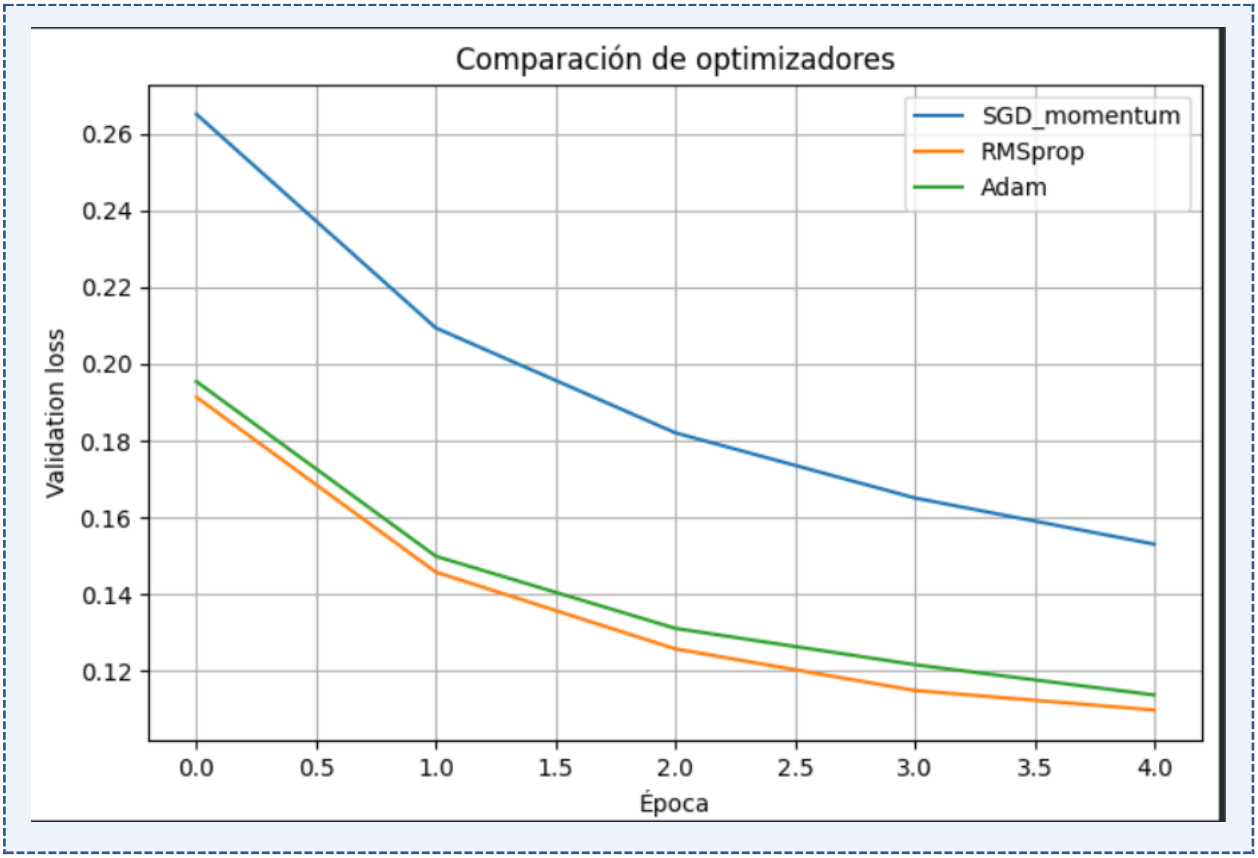


Figura 9. Comparación de la validation loss de SGD, RMSprop y Adam por época

Qué muestra: El desempeño en Dev de cada optimizador bajo condiciones idénticas y la trayectoria de su validation loss.

Qué significa: Las tres curvas de validation loss descienden de forma continua y sin oscilaciones, lo que indica un entrenamiento estable en los tres casos. RMSprop obtuvo la menor validation loss en todas las épocas (≈0,19 a ≈0,11; final 0,1098) y el mayor dev_accuracy (0,9680), seguido muy de cerca por Adam (0,1137 y 0,9673), con una diferencia de apenas 0,0007 en accuracy. SGD con momentum partió de una pérdida notablemente mayor (≈0,27) y terminó en 0,1530 con dev_accuracy de 0,9558; su curva seguía descendiendo al final, por lo que con learning rate 0,01 y solo 5 épocas aún no había convergido. La

conclusión válida es acotada: bajo las condiciones de este experimento (5 épocas, batch 64, learning rates indicados) una configuración mostró mejor o peor comportamiento en Dev; no se concluye que un optimizador sea universalmente superior, y SGD con momentum puede requerir mayor ajuste de su learning rate.

Decisión que permite: Elegir el optimizador para los experimentos siguientes con base en Dev —nunca en Test—. Se elige Adam con learning rate 0,001 para los experimentos siguientes: junto con RMSprop redujo la validation loss más rápido y de forma más estable que SGD con momentum, y la diferencia entre ambos es muy pequeña y podría deberse a la aleatoriedad de una sola ejecución. Se mantiene Adam por coherencia con el baseline y con los modelos posteriores. La decisión se basó únicamente en Dev; Test no se utilizó.

5.2 Exploración del learning rate de Adam

Código 12. Efecto del learning rate (0,0001; 0,001; 0,01)

```
learning_rates = [0.0001, 0.001, 0.01]
resultados_lr = []

for lr in learning_rates:
    keras.backend.clear_session()
    tf.random.set_seed(SEED)

    modelo = crear_modelo_experimento(keras.optimizers.Adam(learning_rate=lr))
    historia = modelo.fit(
        x_train, y_train,
        validation_data=(x_dev, y_dev),
        epochs=3, batch_size=64, verbose=0
    )

    resultados_lr.append({
        "learning_rate": lr,
        "train_accuracy": historia.history["accuracy"][-1],
        "dev_accuracy": historia.history["val_accuracy"][-1],
        "dev_loss": historia.history["val_loss"][-1]
    })

display(pd.DataFrame(resultados_lr))
```

	learning_rate	train_accuracy	dev_accuracy	dev_loss
0	0.0001	0.916188	0.919417	0.288311
1	0.0010	0.969500	0.962417	0.128201
2	0.0100	0.967812	0.958083	0.173259

Figura 10. Tabla de resultados para tres learning rates de Adam

Tabla 11. Resultados por learning rate (3 épocas, transcribir de la captura)

Learning rate	Tamaño del paso	Train accuracy	Dev accuracy	Dev loss
0,0001	Pequeño	0,9162	0,9194	0,2883
0,001	Intermedio	0,9695	0,9624	0,1282
0,01	Grande	0,9678	0,9581	0,1733

Qué muestra: El efecto del tamaño del paso sobre el aprendizaje en tres épocas.

Qué significa: Un learning rate muy alto tiende a producir inestabilidad y uno muy bajo a un aprendizaje lento. Con 0,0001 el aprendizaje fue lento: tras 3 épocas el dev_accuracy llegó solo a 0,9194 y la dev_loss quedó en 0,2883. Con 0,001 se obtuvo el mejor resultado (dev_accuracy 0,9624; dev_loss 0,1282). Con 0,01 el paso ya fue demasiado grande: el accuracy en Dev (0,9581) fue menor y la dev_loss (0,1733) fue mayor que con 0,001, aunque el modelo no llegó a divergir. El valor intermedio ofrece el mejor equilibrio entre velocidad y estabilidad, y se conserva como learning rate de referencia.

Decisión que permite: Fijar el learning rate de referencia para los experimentos posteriores.

Tabla 12. Hiperparámetros relevantes y efecto de valores extremos

Hiperparámetro	Qué controla	Si es extremo
Learning rate	Tamaño del paso.	Muy alto: inestabilidad. Muy bajo: aprendizaje lento.
Batch size	Ejemplos procesados antes de actualizar.	Cambia ruido, memoria y velocidad.
Número de neuronas	Capacidad de representación.	Pocas: underfitting; demasiadas: mayor costo y riesgo de overfitting.
Épocas	Pasadas completas por Train.	Pocas: aprendizaje insuficiente; demasiadas: posible sobreajuste.

6. Regularización y overfitting

Notebook de origen: TallerRedesNeuronalesIII.ipynb. Para hacer visible la diferencia entre memorizar Train y generalizar hacia Dev se utiliza una muestra estratificada de 12.000 ejemplos de Train y una red más grande (512 y 256 neuronas ocultas). Se comparan dos modelos con la misma arquitectura base: uno **sin regularización** y otro con **L2 (1e-4), Dropout (0,30) y Early Stopping** (monitor val_loss, patience=2, restore_best_weights=True). Ambos tienen 535.818 parámetros entrenables.

Código 13. Muestra estratificada de Train

```
x_train_reg, _, y_train_reg, _ = train_test_split(
    x_train, y_train,
    train_size=12000,
    random_state=SEED,
    stratify=y_train
)
print("Muestra:", x_train_reg.shape, y_train_reg.shape)
```

Código 14. Modelo sin regularización (15 épocas)

```
def crear_modelo_sin_regularizacion():
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(512, activation="relu"),
        layers.Dense(256, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
    modelo.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return modelo

modelo_sin_reg = crear_modelo_sin_regularizacion()
hist_sin_reg = modelo_sin_reg.fit(
    x_train_reg, y_train_reg,
    validation_data=(x_dev, y_dev),
    epochs=15, batch_size=64, verbose=0
)
```

Código 15. Modelo con L2, Dropout y Early Stopping (hasta 20 épocas)

```
def crear_modelo_regularizado():
    modelo = keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(512, activation="relu",
                     kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dropout(0.30),
        layers.Dense(256, activation="relu",
                     kernel_regularizer=regularizers.l2(1e-4)),
        layers.Dropout(0.30),
        layers.Dense(10, activation="softmax")
    ])
    modelo.compile(
        optimizer=keras.optimizers.Adam(0.001),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return modelo

parada_temprana = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)

modelo_reg = crear_modelo_regularizado()
hist_reg = modelo_reg.fit(
    x_train_reg, y_train_reg,
    validation_data=(x_dev, y_dev),
    epochs=20, batch_size=64,
    callbacks=[parada_temprana],
    verbose=0
)

print("Épocas ejecutadas:", len(hist_reg.history["loss"]))
```

```
Épocas ejecutadas: 8
```

Figura 11. Número de épocas ejecutadas con Early Stopping

Qué muestra: Aunque se configuraron 20 épocas, el entrenamiento se detuvo en la época 8; la validation loss no mejoró en las épocas 7 y 8 y se restauraron los pesos de la época 6, donde alcanzó su mínimo.

Qué significa: Early Stopping interrumpió el entrenamiento porque la validation loss no mejoró durante 2 épocas consecutivas (patience=2) y restauró los pesos de la mejor época (restore_best_weights=True).

Decisión que permite: Evitar entrenamiento innecesario y conservar el punto de mejor generalización observado en Dev.

6.1 Comparación de curvas

Código 16. Curvas de pérdida con y sin regularización

```
plt.figure(figsize=(8, 5))
plt.plot(hist_sin_reg.history["loss"],     label="Train sin regularización")
plt.plot(hist_sin_reg.history["val_loss"], label="Dev sin regularización")
plt.plot(hist_reg.history["loss"],         label="Train regularizado")
plt.plot(hist_reg.history["val_loss"],     label="Dev regularizado")
plt.xlabel("Época"); plt.ylabel("Loss")
plt.title("Regularización: comparación de pérdidas")
plt.legend(); plt.grid(True); plt.show()
```

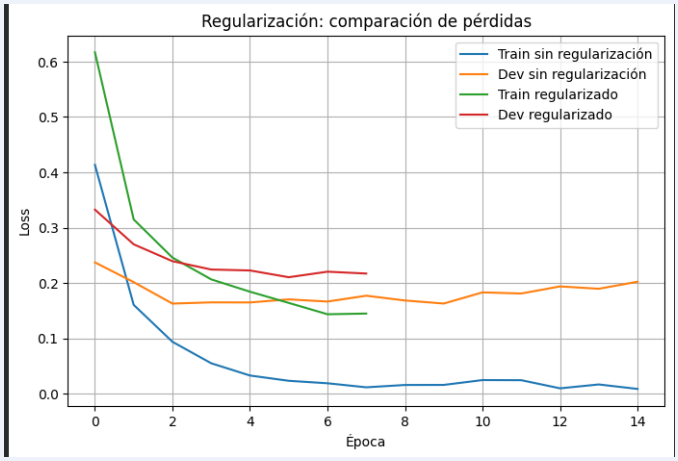


Figura 12. Comparación de loss Train/Dev del modelo sin regularización y del regularizado

Qué muestra: La evolución del loss en Train y Dev para ambos modelos.

Qué significa: El overfitting suele aparecer cuando Train sigue mejorando mientras Dev se estanca o empeora. En el modelo sin regularización, el loss de Train cae hasta ≈0,01 mientras la validation loss alcanza su mínimo hacia la tercera época (≈0,16) y luego se estanca y vuelve a subir (≈0,20 al final), lo que es overfitting: memoriza Train sin ganar generalización. En el modelo regularizado la brecha es menor: Train baja de forma más moderada (≈0,14) y la validation loss se estabiliza alrededor de 0,21–0,22. Nota: el loss del modelo regularizado incluye el término de penalización L2, por lo que su valor absoluto no es directamente comparable con el del modelo sin regularización; lo relevante es la forma de las curvas y la brecha.

Decisión que permite: Decidir si la regularización redujo la brecha y, por tanto, si mejora la generalización.

6.2 Evaluación de ambos modelos en Dev

Código 17. Evaluación en Dev

```
dev_sin_reg = modelo_sin_reg.evaluate(x_dev, y_dev, verbose=0)
dev_reg     = modelo_reg.evaluate(x_dev, y_dev, verbose=0)

print("Sin regularización - dev loss/accuracy:", dev_sin_reg)
print("Regularizado      - dev loss/accuracy:", dev_reg)
```

```
Sin regularización - dev loss/accuracy: [0.20214247703552246, 0.9614166617393494]
Regularizado      - dev loss/accuracy: [0.2103145271539688, 0.9629166722297668]
```

Figura 13. Loss y accuracy en Dev del modelo sin regularización y del regularizado

Tabla 13. Comparación en Dev

Modelo	Dev loss	Dev accuracy
Sin regularización	0,2021	0,9614
Con L2 + Dropout + Early Stopping	0,2103	0,9629

Qué muestra: El desempeño de ambos modelos en datos no usados para actualizar pesos.

Qué significa: La regularización puede reducir un poco el accuracy de Train y aun así mejorar la generalización; por eso no se evalúa mirando solo Train. El modelo regularizado generaliza ligeramente

mejor: su accuracy en Dev fue 0,9629 frente a 0,9614 (+0,15 puntos porcentuales), con un accuracy de Train menor (0,9798 frente a 0,9978), por lo que la brecha Train–Dev bajó de ≈3,6 a ≈1,5 puntos. La mejora en accuracy es pequeña; su principal efecto es reducir la brecha. Su dev_loss (0,2103) es algo mayor que la del modelo sin regularización (0,2021) porque incluye la penalización L2.

Decisión que permite: Seleccionar el modelo regularizado (modelo_reg) como modelo final para la evaluación en Test, porque redujo la brecha Train–Dev y obtuvo un accuracy en Dev igual o mayor que el modelo sin regularización.

7. Evaluación del modelo

Notebook de origen: TallerRedesNeuronalesIII.ipynb. A partir de este punto se usa **Test** para reportar el desempeño final del modelo regularizado; el modelo no se vuelve a ajustar con base en estos resultados. Se calculan Accuracy y Precision, Recall y F1 con promedio macro, además del reporte por clase y la matriz de confusión.

Código 18. Predicciones sobre Test

```
probabilidades_test = modelo_reg.predict(x_test, verbose=0)
y_pred = np.argmax(probabilidades_test, axis=1)

print("Primeras predicciones:", y_pred[:20])
print("Primeras clases reales:", y_test[:20])
```

7.1 Métricas globales

Código 19. Accuracy, Precision, Recall y F1 (macro)

```
accuracy = accuracy_score(y_test, y_pred)
precision_macro = precision_score(y_test, y_pred, average="macro", zero_division=0)
recall_macro    = recall_score(y_test, y_pred, average="macro", zero_division=0)
f1_macro        = f1_score(y_test, y_pred, average="macro", zero_division=0)

metricas = pd.DataFrame({
    "metrica": ["Accuracy", "Precision macro", "Recall macro", "F1 macro"],
    "resultado": [accuracy, precision_macro, recall_macro, f1_macro]
})
display(metricas)
```

	metrica	resultado
0	Accuracy	0.964900
1	Precision macro	0.964538
2	Recall macro	0.964826
3	F1 macro	0.964596

Figura 14. Tabla de métricas globales del modelo final en Test

Tabla 14. Métricas finales en Test

Métrica	Resultado
Accuracy	0,9649
Precision macro	0,9645
Recall macro	0,9648
F1 macro	0,9646

Qué muestra: El desempeño global del modelo final sobre datos que nunca influyeron en su diseño.

Qué significa: El accuracy (0,9649) y las métricas macro (Precision 0,9645, Recall 0,9648, F1 0,9646) son casi idénticas, lo que indica un rendimiento homogéneo entre clases, coherente con un dataset prácticamente balanceado. El accuracy en Test (0,9649) es muy cercano al de Dev (0,9629), por lo que no hay señales de selección excesiva sobre Dev ni de data leakage.

Decisión que permite: Reportar el desempeño esperado del modelo sobre datos nuevos.

7.2 Reporte por dígito

Código 20. classification_report

```
print(classification_report(y_test, y_pred, digits=4, zero_division=0))
```

```
•••              precision    recall  f1-score   support

           0     0.9671    0.9888    0.9778       980
           1     0.9894    0.9833    0.9863      1135
           2     0.9672    0.9438    0.9554      1032
           3     0.9480    0.9743    0.9609      1010
           4     0.9531    0.9735    0.9632       982
           5     0.9534    0.9641    0.9588       892
           6     0.9667    0.9708    0.9688       958
           7     0.9750    0.9484    0.9615      1028
           8     0.9627    0.9548    0.9588       974
           9     0.9627    0.9465    0.9545      1009

    accuracy                         0.9649     10000
   macro avg     0.9645    0.9648    0.9646     10000
weighted avg     0.9650    0.9649    0.9649     10000
```

Figura 15. Reporte de clasificación por dígito

Qué muestra: Las métricas desglosadas por clase.

Qué significa: Una métrica global alta puede ocultar clases débiles. La clase con menor desempeño fue el dígito 9 (Precision 0,9627, Recall 0,9465, F1 0,9545), por debajo del F1 macro de 0,9646. También quedaron por debajo del promedio el dígito 2 (Recall 0,9438, el más bajo de todos; F1 0,9554) y el dígito 3 (Precision 0,9480, la más baja). Una interpretación plausible es la similitud visual entre dígitos: el 9 se confunde con el 4 (trazo superior abierto), el 2 con el 3 y el 8, y el 3 con el 5. Además, una red densa aplana la imagen de 28 × 28 y pierde la estructura espacial, lo que dificulta distinguir trazos parecidos.

Decisión que permite: Identificar en qué dígitos debe concentrarse una mejora.

7.3 Matriz de confusión

Código 21. Matriz de confusión

```
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(8, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.arange(10))
disp.plot(ax=ax, cmap="Blues", colorbar=False)
plt.title("Matriz de confusión - MNIST")
plt.show()
```

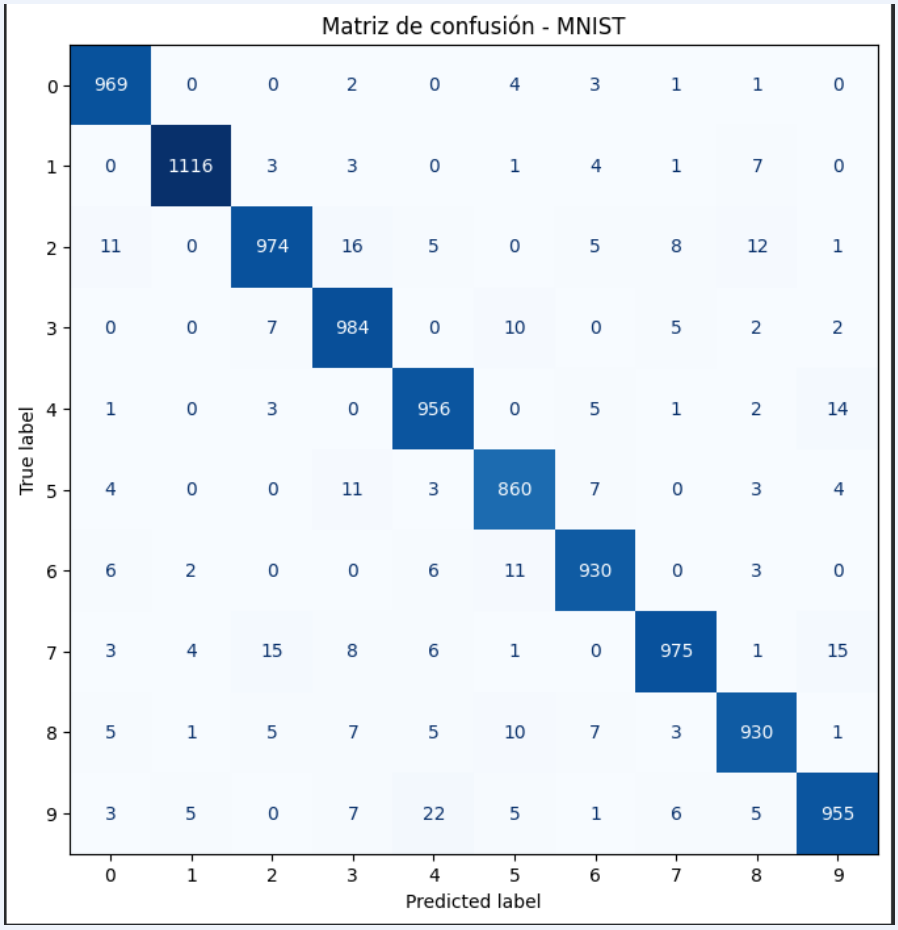


Figura 16. Matriz de confusión del modelo final sobre Test

Qué muestra: Cuántos ejemplos reales de cada dígito se predijeron como cada clase posible; la diagonal representa los aciertos.

Qué significa: La diagonal concentra la gran mayoría de los aciertos. Los pares más confundidos fueron 9 clasificado como 4 (22 veces), 2 como 3 (16), 7 como 2 (15) y 7 como 9 (15). La confusión no es simétrica: el 9 se clasificó como 4 en 22 casos, pero el 4 como 9 en 14. También se observan confusiones entre 3 y 5 (10 y 11 veces) y de 2 como 8 (12).

Decisión que permite: Orientar el análisis de errores hacia los pares de clases problemáticos.

7.4 Confusión más frecuente y error concreto

Código 22. Confusión más frecuente y ejemplo mal clasificado

```
cm_errores = cm.copy()
np.fill_diagonal(cm_errores, 0)

real_confundido, predicho_como = np.unravel_index(
    np.argmax(cm_errores), cm_errores.shape
)
cantidad = cm_errores[real_confundido, predicho_como]

print(f"La confusión más frecuente fue: {real_confundido} "
      f"clasificado como {predicho_como} ({cantidad} veces)")

indices = np.where((y_test == real_confundido) & (y_pred == predicho_como))[0]
if len(indices) > 0:
    idx = indices[0]
    plt.imshow(x_test[idx], cmap="gray")
    plt.title(f"Real: {y_test[idx]} | Predicción: {y_pred[idx]}")
    plt.axis("off")
    plt.show()
```

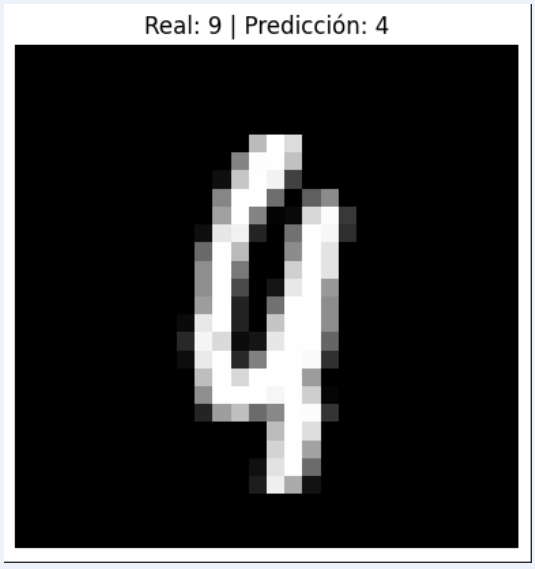


Figura 17. Confusión más frecuente e imagen de un ejemplo mal clasificado

Qué muestra: El par de clases con más errores —el dígito 9 clasificado como 4, 22 veces— y un ejemplo real de ese error.

Qué significa: En la imagen mostrada, un 9 real fue predicho como 4: el trazo superior del dígito queda abierto y sus dos líneas verticales con el cruce en la parte media son morfológicamente muy parecidas a un 4 manuscrito, por lo que el error resulta comprensible incluso para una persona. El objetivo no es ocultar los errores, sino entender dónde falla el modelo.

Decisión que permite: Proponer mejoras concretas: arquitecturas convolucionales que aprovechen la estructura espacial, aumento de datos o más épocas con regularización.

8. Diagnóstico de bias y variance

Notebook de origen: TallerRedesNeuronalesIII.ipynb (Laboratorio E). El diagnóstico se hace observando conjuntamente Train y Dev; no se asigna la etiqueta de bias o variance por una sola cifra. Se compara un modelo intencionalmente pequeño (16 neuronas, 2 épocas) con el modelo sin regularización y con el regularizado.

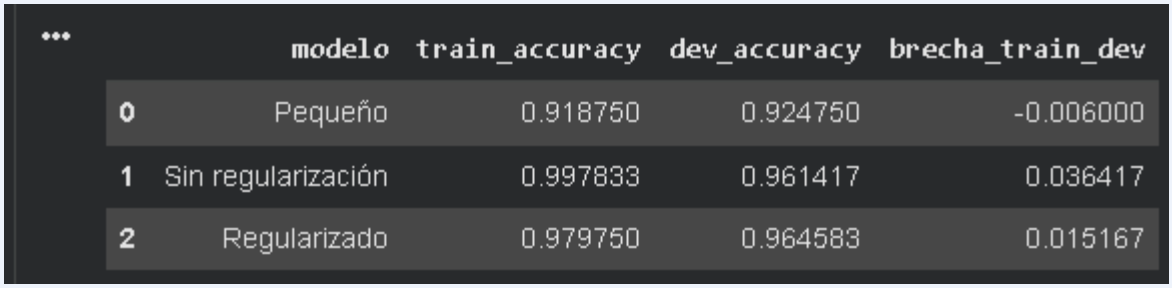
Código 23. Modelo intencionalmente pequeño

```
modelo_pequeno = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(16, activation="relu"),
    layers.Dense(10, activation="softmax")
])
modelo_pequeno.compile(
    optimizer=keras.optimizers.Adam(0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
hist_pequeno = modelo_pequeno.fit(
    x_train, y_train,
    validation_data=(x_dev, y_dev),
    epochs=2, batch_size=64, verbose=0
)
```

Código 24. Resumen de la brecha Train–Dev

```
def resumen_historia(nombre, historia):
    train_acc = historia.history["accuracy"][-1]
    dev_acc = historia.history["val_accuracy"][-1]
    return {
        "modelo": nombre,
        "train_accuracy": train_acc,
        "dev_accuracy": dev_acc,
        "brecha_train_dev": train_acc - dev_acc
    }

tabla_diagnostico = pd.DataFrame([
    resumen_historia("Pequeño", hist_pequeno),
    resumen_historia("Sin regularización", hist_sin_reg),
    resumen_historia("Regularizado", hist_reg)
])
display(tabla_diagnostico)
```



	modelo	train_accuracy	dev_accuracy	brecha_train_dev
0	Pequeño	0.918750	0.924750	-0.006000
1	Sin regularización	0.997833	0.961417	0.036417
2	Regularizado	0.979750	0.964583	0.015167

Figura 18. Tabla de diagnóstico Train–Dev (accuracy y brecha)

Tabla 15. Diagnóstico por modelo

Modelo	Train acc.	Dev acc.	Brecha	Diagnóstico
Pequeño	0,9188	0,9248	−0,0060	Bias alto (underfitting)
Sin regularización	0,9978	0,9614	0,0364	Variance alta (overfitting)
Regularizado	0,9798	0,9646	0,0152	Mejor equilibrio

Tabla 16. Criterios de interpretación y acción

Patrón	Lectura posible	Acción
Train y Dev bajos	Bias / underfitting	Más capacidad, más entrenamiento o mejores características.
Train alto y Dev bastante menor	Variance / overfitting	Regularización, más datos, simplificar o Early Stopping.
Train y Dev altos y cercanos	Mejor equilibrio	Confirmar con Test y métricas por clase.
Dev fluctúa mucho	Entrenamiento inestable	Revisar learning rate, batch size o inicialización.

Qué muestra: Los niveles de accuracy en Train y Dev y su diferencia para tres capacidades de modelo.

Qué significa: El modelo pequeño (16 neuronas, 2 épocas) tiene ambos accuracy bajos (≈0,92), lo que sugiere bias alto: no captura suficiente patrón. Su brecha es ligeramente negativa porque el accuracy de Train se promedia durante la época, cuando los pesos aún mejoran. El modelo sin regularización alcanza 0,9978 en Train pero solo 0,9614 en Dev (brecha de 3,6 puntos): variance alta. El regularizado reduce la brecha a 1,5 puntos con el mayor accuracy en Dev (0,9646 en la última época del historial; 0,9629 al evaluar con los pesos restaurados), lo que indica el mejor equilibrio. El tradeoff se observa al aumentar la capacidad: se reduce el bias pero aparece variance, que la regularización controla.

Decisión que permite: Definir si conviene aumentar capacidad, regularizar más o mantener la configuración actual.

9. Conclusiones

Qué funcionó

- La partición estratificada en Train/Dev/Test y la normalización permitieron comparar configuraciones sin usar Test para decidir; el accuracy en Test (0,9649) fue consistente con el de Dev (0,9629), sin señales de data leakage.
- En la comparación de optimizadores, RMSprop obtuvo el mejor desempeño en Dev bajo las condiciones del experimento (dev_accuracy 0,9680), seguido de cerca por Adam (0,9673); SGD con momentum quedó por detrás (0,9558). Se continuó con Adam (learning rate 0,001), que fue también el mejor valor explorado (dev_accuracy 0,9624 en tres épocas).
- La combinación de L2, Dropout y Early Stopping redujo la brecha Train–Dev frente al modelo sin regularización (de ≈3,6 a ≈1,5 puntos de accuracy); Early Stopping detuvo el entrenamiento en la época 8, restaurando los pesos de la época 6.
- El modelo final alcanzó en Test un accuracy de 0,9649 y un F1 macro de 0,9646.

Limitaciones

- Los resultados provienen de una sola ejecución con una semilla; la aleatoriedad (inicialización, Dropout, hardware de Colab) puede producir variaciones, por lo que las diferencias pequeñas entre configuraciones deben interpretarse con cautela.
- Los experimentos usaron pocas épocas y una búsqueda de hiperparámetros limitada (tres optimizadores y tres learning rates).
- La red densa aplana la imagen y no aprovecha la estructura espacial; la confusión de 9 como 4 (22 casos) es un ejemplo de esta limitación.

Mejoras propuestas

- Repetir los experimentos con varias semillas y reportar media y dispersión.
- Explorar batch size, número de neuronas, tasa de dropout y fuerza de L2 de forma sistemática, además de L1 y programación del learning rate.
- Utilizar redes convolucionales y aumento de datos para reducir los errores entre dígitos visualmente similares.

10. Repositorio en GitHub

El notebook ejecutado, con sus salidas visibles, y este informe se encuentran publicados en el siguiente repositorio:

Enlace: https://github.com/NicolasRomero22/Redes-Neuronales-III

Estructura del repositorio:

```
Redes-Neuronales-III/
├── TallerRedesNeuronalesIII.ipynb
├── Informe_Redes_Neuronales_III.pdf
└── README.md
```

El archivo README.md incluye el objetivo, el dataset (MNIST), los requisitos (Python, TensorFlow/Keras, scikit-learn, pandas, matplotlib), el orden de ejecución del notebook y el enlace al informe en PDF. El notebook puede ejecutarse en Google Colab con «Ejecutar todo» para reproducir los resultados.

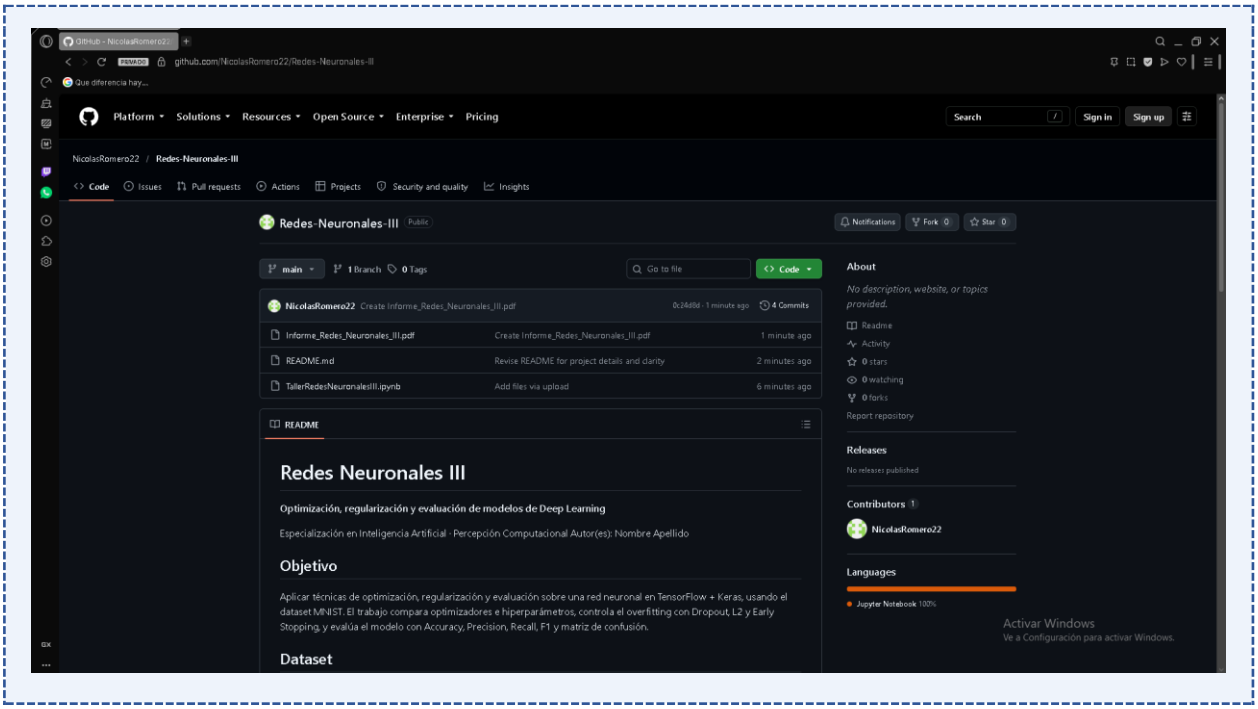


Figura 19. Repositorio de GitHub con los notebooks y el README

Referencias

CADI Deep Learning – Conceptos (2026). Guía completa de trabajo y laboratorio: Redes Neuronales III. Especialización en Inteligencia Artificial.